python-gpu-maximize

The new standard for running Python at maximum speed on any consumer box -- CPU and GPU together, evidence-first. Program overview, specifications, stack recipe, and user flow. Thomas Jarvis home lab · 2026-07-28 (rev. current running config).

1. What We Are Doing

We are building a universal execution harness that takes any Python script and runs it at the fastest speed the machine can honestly deliver, using CPU and GPU concurrently where proof supports it. It was extracted from a production trading-research stack (the "rolex" gauntlet: 2,772-task simulation runs) where every feature earned its place by fixing a real failure: runs dying with closed terminals, unresumable half-finished work, per-row disk barriers silently throttling 16 cores, and GPU dispatch that made code slower.

The four covenants (the product identity): 1. Never slower than naive invocation -- ambiguous work routes to CPU. 2. An accelerator is promoted only after transfer-inclusive, end-to-end crossover and parity are proven for the actual workload shape. 3. Progress reporting states only durable truth -- no fake percentages, no in-memory counters that drift from disk. 4. Every long run is detached and resumable by default -- a closed laptop lid never costs work again.

Why this niche: surveyed and slim. Existing tools either assume GPU-always-wins, are framework-locked, or ignore durability entirely. Nothing combines conservative auto-routing + resource governance + task-level resume + warm GPU state on consumer hardware.

2. Specifications

2.1 Execution modes

Mode | Trigger | Behavior

Teach | Double-click / zero args | Persistent terminal opens showing grouped flag help + quickstart. Never executes a workload.

maximize script.pymaximize --headless script.py

2.2 Flag system (teaching-grade -h)

Group | Flags | Notes

Routing | auto = AST inspection, conservative; forcing a backend is for proven crossovers only

--backend {auto,cpu,torch,cupy,numba}, --device 0,1--vram 8G | --vram 60% | --vram 0:8G,1:4G, --ram 24G--workers N, --reserve-threads N--fsync {per_row,batched,interval}, --state-dir, --resume RUN_ID--dry-run, --profile [s], --quiet, --json

2.3 Status bar (TTY only, auto-suppressed in pipes)

[####------] 412/1,000 tasks | 3.9/min | ETA 02:31 | CPU 62% GPU0 6.2/8.0G | RUN_ID

Sourced exclusively from the durable rows/receipts stream the resume scanner trusts; rate/ETA from a ~5-minute trailing window. No task count exposed -> honest degradation to elapsed + utilization.

2.4 Resume & durability

Task-level resume across arbitrary directory nesting (incl. physical shards): scans durable rows.jsonl completion streams, validates task identity, set-subtracts against the expected matrix, refuses conflicts fail-closed. Proven repeatedly under fire: across a multi-death production night it reconstructed completed work exactly (e.g. 1,786/2,772 retained matching preflight to the row) and dry-ran clean against live runs. A new bounded-memory offline finalizer now closes the last gap -- a completed corpus whose in-process aggregation would OOM the host is finalized off-line with byte-identical fingerprint parity to the legacy path (external merge == independent SQLite BINARY hash), peak RSS under a hard 256 MiB gate on a real 1.05M-row trace. The durable_jsonl writer: measured root cause was per-row os.fsync() -- 8.87 ms/write barriers on SATA (1.59 ms NVMe) starving 16 cores at 741 IOPS; fsync policy became a mode, hot/durable directory split became config.

2.5 Warm GPU service

Loopback-only authenticated RPC daemon owning one CUDA context: typed array bytes (never pickle), SHA-bound

TorchScript model cache, idle VRAM eviction. Kills the 448 ms cold-build / 63.6 ms cold-kernel tax -- paid once per boot, not per script. Generalizes to any "keep expensive state warm" payload.

3. Stack Recipe

Layer	Reference build (this box)	Universal tier
Hardware	RTX 5070 Ti 16 GB (sm_120), 16-thread CPU, 63 GB RAM, NVMe hot + SATA durable	Any consumer box; CPU-only is a first-class tier (numba, threads, affinity, shared memory)

PYTHON_GPU_*_PYTHON) for split setups

Hard-won traps encoded in the router

- LightGBM device_type='cuda' silently runs CPU on Windows -> tree models stay CPU-batched.

4. User Design Flow

First contact (90 seconds): double-click -> terminal teaches the flag system -> maximize --dry-run my_script.py shows the routing decision + device table, touches nothing -> maximize my_script.py runs foreground with the status bar. Production: maximize --headless --workers 14 --vram 70% --fsync interval script.py -> detached run, receipts + checkpoints on disk -> box reboots mid-run -> maximize --resume RUN_ID script.py -> completed tasks skipped, remainder dispatched, identity validated fail-closed. Automation / agents: same flags + --json + documented exit codes -> CI, cron, and LLM subagents drive it blind from one command line.

5. Architecture Diagram

any_script.py
+ flags (or defaults) v AST Router
inspect, never execute -> CPU if ambiguous v Admission Governor
measured-free VRAM/RAM/CPU vs budgets v Detached Supervisor
launch once · thread budget · receipts · py-spy · status bar (TTY) / silent (headless) v CPU Workers
N lanes = cores - reserved GPU Backend
torch/cupy/numba · VRAM ceiling Warm GPU Service
loopback RPC · warm CUDA + models v Durable Truth Layer
durable_jsonl (per_row/batched/interval) · receipts · task-level RESUME -- resume reads this, never memory

6. Why This Is a True Game Changer